MDPI

Article

HARK: Human-Accountable, Authority-Typed Review Knowledge for Evidence-Bearing Document Review

Zhitong Yang [1], Qingxian Wang [1], Kai Wang [1], Qi Li [1], Lin Song [1], Dahai Jin [1], Hong Wang [1]

[1] State Grid Customer Service Center, Tianjin 300300, China
* Correspondence: yyyyyyzt@tju.edu.cn

Abstract

Rule-governed document review requires explicit criteria, source-addressable evidence, and a distinction between provisional findings and accountable human disposition. We present HARK (Human-Accountable Review Knowledge), an architecture and executed system that represents review criteria as rules with hard, hybrid, or soft operational authority. Deterministic verifiers produce checks and candidate spans; a positional evidence gate checks submitted quotations; a policy derives suggested verdicts; and decision records support feedback storage. On a fixed corpus of 2,000 proposals with 5,323 planted defects, the combined configuration achieves micro-F1 0.746 versus 0.205 for a keyword baseline and 75.4% policy-verdict agreement. Among 1,612 policy-non-Pass proposals (expected Revise or Fail), 493 receive less severe suggestions (under-calls); 319 receive Pass: 152 expected Revise and 167 expected Fail, the latter among 799 expected Fail proposals. A total of 1,744 of 2,158 misses enter pending review. HARK thus provides an executable architecture linking rule-specific authority, located evidence, and recorded human disposition. Deployment-level approval and recovery requirements are specified separately from the fixed-corpus evaluation.

Keywords: document review; review knowledge representation; operational authority; evidence traceability; deterministic verification; human disposition; feedback persistence

1. Introduction

Rule-governed, evidence-bearing document review combines three conditions: explicit review criteria, a source-addressable document, and a distinction between provisional findings and accountable human disposition. Checklist-based inspection provides a foundation for this task [1,2]. The computational problem is not only to generate comments, but to represent which criteria a system can check, what evidence it can supply, and which decisions remain with a reviewer. We study technical solution proposals for a large utility's online service platform: documents describing intended architecture, interfaces, data flows, implementation plans, and risks before implementation. Compliance dossiers and architecture designs are potential applications of the same task formulation. Explicit review criteria also arise outside internal engineering practice: the EU Artificial Intelligence Act, for example, conditions market access for high-risk systems on documented risk management, technical documentation and conformity assessment [3], which makes the inspectability of review criteria a regulatory as well as an engineering concern.

The motivating prototype represented 24 expert rules through keyword lists and prompts, without a persistent feedback layer. This arrangement made it difficult for reviewers to distinguish a requirement's mention from its satisfaction or to connect a

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Computers* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

proposed finding to an inspectable decision path. Its keyword component is represented by baseline B0 in the evaluation. On the fixed corpus, B0 recovers 16.4% of planted defects, produces 2,313 false findings against 871 true findings, and achieves policy-verdict agreement of $\kappa = 0.034$. These results motivate a different representation and routing architecture; they do not identify model failure or isolate the causal contribution of typing.

Consider the criterion that a proposal specify load testing before go-live. "No load test is planned" and "Load testing plan: to be determined" mention the activity without supplying an adequate plan. Conversely, "A pre-production capacity drill will measure response time at 500 TPS before go-live" may provide relevant evidence without the phrase "load test". These illustrative formulations separate lexical presence, document location, and substantive satisfaction. A deterministic cue can support a limited check or identify a passage for review; it does not by itself establish that a testing plan is sufficient.

We call the representation and routing task the *review-knowledge ingestion problem*: how should review criteria, evidence expectations, and decision permissions enter an assisted review system? Keyword checks offer reproducible surface tests; prompts convey contextual instructions; retrieval supplies relevant prior material. None of these mechanisms alone specifies the complete contract linking a criterion, a document range, a proposed finding, and its disposition. HARK combines them through explicit rule records and review states rather than treating the model's output as the final decision.

Two design tensions guide the architecture. First, review criteria mix observable structure with contextual judgement: section presence can be checked mechanically, whereas risk adequacy requires interpretation. Verifiability is therefore relevant to routing [4,5], but the ability to execute a lexical test must not be confused with complete decidability of its business requirement. Second, useful decision history may inform later reviews, yet recording a decision, authorising a knowledge change, and demonstrating improved review quality are different accomplishments. A system must expose these boundaries rather than infer human approval from a name or infer learning benefits from stored feedback.

We present HARK (Human-Accountable Review Knowledge) and examine its design, evaluated implementation snapshot, and validation boundaries. Accountability here refers to recorded human disposition and an inspectable knowledge state rather than an enforced identity-authentication guarantee. The system is realised as an extension of a general-purpose agent host: a host-independent review engine carries the representation and decision logic, while a thin host integration supplies storage, tools and delivery. This separation makes the architecture reusable beyond a single deployment. The paper makes four contributions:

(1) A task formulation for rule-governed, evidence-bearing document review and a rule representation that separates evidence production, provisional findings, suggested verdicts, and human disposition (Section 3.1).
(2) An operational authority model with hard, hybrid, and soft routes, verifier-produced absence records, quotation checks, and a deterministic verdict policy (Sections 3.2–3.5).
(3) A host-independent engine with a thin deployment integration, carrying feedback records, overrides, exemplars and review-state storage, together with deployment contracts for authenticated approval, comprehensive effective-authority control, and content-addressed recovery (Sections 3.6–3.8).
(4) A fixed end-to-end regression comparison, a traceability and pending-workload analysis, and development-team workflow evidence from internal proposals, within an operation whose first year covered approximately 2,000 operational documents (Section 4).

https://doi.org/10.3390/computers1010000

Four research questions organise the evaluation. **RQ1:** How are review knowledge and operational authority represented, and where are the implementation boundaries? **RQ2:** What detection and verdict trade-offs do the end-to-end configurations exhibit on the fixed corpus? **RQ3:** What traceability and gate-acceptance evidence is available, and what unresolved workload is routed to human review? **RQ4:** What workflow evidence supports decision persistence, and which deployment controls are required for authenticated approval and version recovery? RQ2 concerns a combined-configuration comparison rather than the causal effect of typing; RQ4 addresses workflow and recovery evidence rather than longitudinal quality improvement.

Figure 1 outlines the architecture. A Markdown document is parsed into source-addressable structures; verifier checks produce findings or candidate spans; a judge submits quotations; a gate checks their location; and a deterministic policy derives a suggested verdict from eligible findings. Decisions and feedback are stored separately. The evaluated snapshot implements the document-processing path and a located-overlap gate; Section 3.7 separately specifies the deployment contracts for authenticated approval, comprehensive authority control, and configuration recovery.

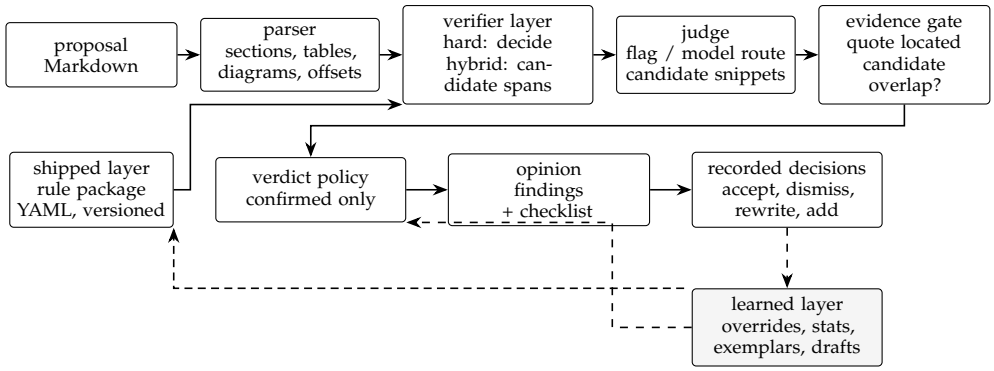


Figure 1. Review architecture and feedback flow. Solid arrows depict document processing; dashed arrows depict decision and knowledge-update paths. The evaluated prototype implements the processing path, a located-overlap gate, and stored feedback for inspection and audit. Section 3.7 specifies the deployment contracts for authenticated approval, comprehensive authority control, and immutable configuration snapshots.

2. Related Work

Rule-governed, evidence-bearing document review.

Document inspection has a long history of using explicit review procedures and checklists [1,2]. Requirements engineering adds quality criteria, smell detection, and traceability to this tradition [4–6]. We use *rule-governed, evidence-bearing document review* to denote a task in which an organisation supplies review requirements, a system associates proposed findings with evidence in the submitted material, and an accountable reviewer resolves issues that the automated procedure cannot settle. Requirements specifications, policy-governed contracts, and technical solution proposals provide different instances of this task [7–9]. Security requirements and industrial certification research further illustrate the importance of explicit review criteria [10,11], while traceability has also been studied for an in-the-wild augmented-reality learning service [12]. These sources motivate the broader task setting; their existence does not establish that HARK transfers across those domains. Technical solution proposal review remains the only empirical case in this study.

Requirements quality and operational verifiability.

Requirements-smell analysis detects cues such as weak wording, placeholders, and missing units [6]. Veizaga et al. combine smell detection with recommendations based on pre-trained language models [13], and AirReq applies smell detection and elimination to commercial aircraft system requirements [14]. HARK draws on these techniques for structural checks and candidate localisation, but a surface cue is not by itself a complete assessment of whether a requirement is satisfied. A further distinction concerns the meaning of *verifiability*. As a requirements-quality attribute, verifiability concerns whether satisfaction can be established through an appropriate verification procedure [4,5]; it does not mean that the current document parser can decide the requirement. HARK's tiers are operational assignments relative to the available parser, verifier, representation, and rule package. A requirement requiring measurements, external records, or an image may be verifiable in principle while remaining outside this prototype's deterministic checking capability. The tier therefore specifies output eligibility within an implementation, not an intrinsic quality ranking of the requirement.

LLM-assisted requirements review and evaluation.

Reviews of LLM applications in requirements engineering describe uses across extraction, analysis, and specification [15]; a dual-perspective review distinguishes using LLMs for code verification from verifying their generated outputs [16]. MARV uses multiple role agents and retrieval of standards for software requirements specification validation [7]. ReqPlan-Eval explicitly couples evidence-aware routing with human requirements-to-planning decisions and discloses the limits of its evaluation [8]. These approaches supply relevant precedents for evidence use and reviewer involvement. The inspected MARV material does not establish the detailed behaviour of document-range validation or persistent feedback, so we do not infer that those capabilities are absent.

Hallucination research motivates checking generated claims [17], but its findings should not be converted into a universal claim that models cannot perform deterministic tasks. Long-context position effects [18], limits of intrinsic reasoning self-correction without external feedback [19], and model-contingent polarity in a hotel-review annotation task [20] concern distinct failure modes and settings. They motivate separating model judgement from permission to issue a result, rather than proving the effectiveness of HARK's particular routing policy. Likewise, the evaluation literature [21] motivates reporting the corpus, judge, and task conditions under which a mechanism was tested. HARK's deterministic-judge evaluation addresses this bounded mechanism question, not the capabilities of arbitrary deployed LLMs.

Neuro-symbolic policy review.

Neuro-symbolic integration offers several ways for symbolic components to constrain learned components [22]. PolicyGuard compiles organisational policies into typed relational rules, uses LLM extraction to ground atoms from evidence, and applies symbolic rule evaluation [9]. Its v1 report also documents legal-expert refinement during review-engine construction and validation: Appendix G describes editing extraction questions, correcting rule-combination logic, and treating a triggered finding as a case-specific exception while preserving the audit trail. PolicyGuard thus reports expert corrections during construction and validation; our comparison concerns the role of the evidence-producing component and the lifecycle of review knowledge. PolicyGuard combines atom grounding, symbolic evaluation, and local expert corrections, whereas HARK studies rule-specific eligibility for automatic findings and a stored review-feedback layer. These architectural choices support different review workflows.

Galitsky and Rybalov study symbolic checking of LLM-generated fault explanations and control actions against physical and sensor constraints in *process control* [23]. That setting illustrates constraint-based rejection rather than design-document review. HARK's related design choice is to make the eligible output and its evidence conditions explicit for each rule before reviewing a document.

Industrial review and editable feedback knowledge.

Code review provides adjacent evidence about learning from reviewer reactions, although its artefacts and evaluation units differ from document review. Pornprasit and Tantithamthavorn compare fine-tuning and prompting for code review automation [24]. BitsAI-CR's v1 architecture contains *two fine-tuned LLMs*, RuleChecker and ReviewFilter; RuleChecker is not a deterministic rule engine. Its data flywheel updates review rules and their taxonomy as well as supplying data for model improvement [25]. The RovoDev report evaluates a deployed reviewer using offline and online measurements and developer feedback [26]. Lin et al. analyse developer replies to CodeRabbit reviews, reporting rejection and misalignment within the reply-bearing sample rather than establishing rejection rates for all generated comments [27].

CodeRabbit's official Learnings documentation, as accessed on 5 November 2026, describes stored user-generated learnings that can be viewed and edited, with role-governed deletion through the interface and removal requests through chat [28]. These controls provide a concrete precedent for inspectable review knowledge. The documentation does not assess complete historical rollback, permission-ceiling enforcement, or replay of the exact knowledge content used for an earlier review. HARK's stored feedback is therefore positioned against these specific mechanisms rather than against an assumed homogeneous category of opaque industrial agents.

Grounding, non-detection, and pending decisions.

Attributed generation links claims to cited sources [29]. Claim-level evaluation of verbatim citation distinguishes quotation compliance from whether the quoted passage actually supports the claim [30]. HARK adds a rule-associated, tool-side eligibility check: the evaluated prototype locates a submitted quotation and checks overlap with verifier-provided candidate evidence for that rule. This *located-overlap gate* is narrower than a guarantee of complete span containment or semantic entailment. A located quotation can still be irrelevant or misinterpreted, and candidate retrieval can miss relevant material.

Non-detection also needs an explicit evidence boundary. A verifier-owned non-detection record reports that a specified procedure did not find expected material in its declared checking scope; it is not proof that the requirement is unsatisfied in the world, or that relevant content is absent from every representation of the document. HARK reserves such records for verifier output rather than accepting a model-generated assertion of absence. When a hybrid rule lacks usable candidates, or a proposed finding does not meet its evidence conditions, an explicit *pending* state preserves the unresolved obligation instead of turning it into either a pass or a confirmed failure. Documents remain untrusted inputs: prompt-injection attacks against LLM judges [31] motivate this separation, but an evidence gate alone does not establish resistance to injection or protect every write path.

Human control, memory, and version requirements.

Human-in-the-loop research distinguishes learning arrangements according to how control is allocated between system and expert [32]. HARK adopts this control question at the level of review rules and feedback events; the presence of a named reviewer or a stored approval text is not equivalent to authenticating the approving human. Agent-memory research [33], self-refinement [34], reflection-based memory [35], and self-modifying

agents [36] show multiple ways to reuse interactions or improve behaviour. Neither prompting nor retrieval inherently precludes audit logs, persistent consolidation, or versioned knowledge. Those properties depend on the surrounding storage, update, and permission mechanisms.

Memory extraction can expose sensitive stored content [37]. HARK's structured feedback records and short exemplars make stored fields explicit, but privacy requires additional controls beyond this representation. More generally, bounded evolution requires separately establishing the identity and authority of the approving actor, rejecting updates that exceed an authorised permission ceiling, and retaining the actual content needed to replay a prior knowledge state. The evaluated prototype provides operational tier routing, a located-overlap gate, and stored feedback; Section 3.7 specifies the additional deployment controls for human authentication, permission-ceiling enforcement, and content replay. This study reports feedback storage and version labels as implemented mechanisms and does not evaluate longitudinal improvement.

Positioning and contribution boundary.

Table 1 compares concrete reports or explicitly labelled family illustrations rather than scoring entire research traditions. The novelty lies in the composition of *rule-specific output eligibility*, *verifier-owned non-detection records*, *explicit pending outcomes*, and *stored feedback with version requirements* within one review workflow, not in the three tier labels themselves. The measurements reported later concern the implemented proposal-review prototype and its stated judge; broader document classes and the deployment-level human-authorised lifecycle require separate evaluation.

3. Materials and Methods

3.1. Task Formulation and Scope

We study *rule-governed, evidence-bearing document review*. An instance consists of a source-addressable document d, an explicit rule package K, a policy P for aggregating findings, and an accountable reviewer. The system prepares rule-linked findings and unresolved items; the reviewer determines their disposition and the final opinion. This task is broader than technical proposal review but narrower than arbitrary document understanding: it requires explicit criteria, identifiable evidence sources and a meaningful distinction between a machine recommendation and an accountable decision. A source range identifies what was read, not whether its contents are true or sufficient.

The empirical instance is the preliminary review of Chinese technical solution proposals before construction. Its primary business object is the *adequacy of the proposed design*, not proof that the proposed implementation has already been deployed. A rule must specify the lifecycle stage, applicability, expected evidence and predicate being checked. We retain the released package and labels for the retrospective evaluation rather than silently relabelling them to fit this formulation. Where its predicates conflate design and implementation, or parser support with business compliance, the discrepancy is reported and must be resolved before a revised package is evaluated.

Four requirements guide the design: rule-linked evidence and explicit non-detection records; rule-specific eligibility to contribute a finding; a visible distinction between confirmed findings, unresolved items and final disposition; and inspectable feedback with controlled activation and recoverable configuration. These are design requirements, not four completed implementation guarantees. Table 2 separates the evaluated prototype from the stronger deployment contract.

Table 1. Mechanism-level positioning using named reports and versions. "Not assessed" describes the inspected material rather than the absence of a capability. Rows are not a common-benchmark comparison, and reported controls do not by themselves demonstrate complete rollback or human authentication.

Report or illustration	Knowledge and decision mechanism	Evidence and unresolved outcomes	Feedback and demonstrated control boundary
2025 keyword prototype (reimplemented baseline)	Substring checks per proposal rule	Keyword matches without satisfaction semantics	No persistent feedback in this baseline implementation
Requirements-smell family illustration [6,13,14]	Cue detection; some systems add learned recommendations	Requirement-level quality cues; coverage depends on the analyser	No uniform family-wide lifecycle or rollback claim
MARV, 2025 thesis [7]	Role agents with retrieved standards	Requirements validation; detailed document-range gating not assessed	Persistent feedback and rollback not assessed in the inspected material
ReqPlan-Eval, 2026 report [8]	Evidence-aware routing; final human decisions	Evidence records and disclosed evaluation limits	Knowledge evolution not reported as a tested mechanism
PolicyGuard, arXiv v1 [9]	Typed relational policy; LLM-grounded atoms; symbolic evaluation	Atom-level evidence and rule traces	Construction/validation expert corrections and audit trail; deployed rollback not assessed
BitsAI-CR, arXiv v1 [25]	Fine-tuned LLM RuleChecker and ReviewFilter	Generated comments and learned secondary validation	Data flywheel refines models and rule taxonomy; per-reviewer rollback not assessed
RovoDev, arXiv v2 report [26]	Deployed code-review generation and filtering	Comment quality evaluated offline and online	Developer feedback evaluated; knowledge-history replay not assessed
CodeRabbit Learnings, official documentation accessed 5 Nov. 2026 [28]	Stored user-generated review knowledge	Knowledge applied to subsequent code reviews	View/edit/delete controls; complete historical rollback not assessed
HARK evaluated prototype	Operational tier routing and rule-specific output eligibility	Located-overlap gate; verifier non-detection; explicit pending	Stored feedback; deployment contracts for authenticated approval, authority ceilings, and content replay in Section 3.7

Table 2. Design requirements, evaluated capabilities, and remaining deployment requirements.

Requirement	Evaluated prototype	Remaining deployment requirement
Evidence traceability	Rule identifiers, source ranges, verifier absence records and recorded gate outcomes	Separate source location, scope relation and semantic support; validate converted inputs
Rule-specific eligibility	Hard/hybrid/soft routing and package validation	Validate effective authority after every update, not only the tier field
Accountable disposition	Confirmed/pending states, deterministic recommendation and recorded decisions	Host-authenticated approval; explicit disposition or waiver of pending items before signing
Feedback and recovery	Overrides, statistics, exemplars, audit records and an override-count version string	Content-addressed immutable configurations, approval binding and demonstrated restoration

3.2. Operational Authority Types

Each YAML rule declares an identifier, dimension, severity, tier, question, rationale, remediation, applicability condition, verifier specification and status. A shared lexicon supplies heading aliases, placeholder and negation tokens, and keyword families. The package contains 28 rules: 10 hard, 13 hybrid and 5 soft. Seven have applicability preconditions;

the lexicon contains 25 keyword families, 11 heading groups, 12 placeholder tokens and 11 negation tokens.

The tier is an *operational routing declaration*. A hard rule uses a decisive implemented predicate; a hybrid rule combines deterministic checks or candidate localisation with a judgement step; a soft rule raises an unresolved item. Decisive execution of a predicate does not prove that the predicate fully captures the business requirement. For example, a lexical declaration of a load-test plan does not establish its scope, responsibilities, timing or performance criteria. Assigning a hard tier to that proxy cannot make those unmeasured properties automatically verifiable. Domain owners must first define the requirement and then specify which subpredicates the verifier actually covers.

Table 3. Default routes in the evaluated package. These routes describe finding eligibility under the frozen configuration, not authentication of human approval.

Tier	Verifier role	Default outcomes	Finding route
hard	Decisive configured predicate	pass, fail, not applicable	Verifier-generated finding; model registration rejected
hybrid	Structural checks and candidate localisation	structural result, candidate, pending, not applicable	Structural failure may be automatic; otherwise a gate-accepted judgement may count
soft	No verifier	unresolved item	Model registration remains pending until an explicit disposition

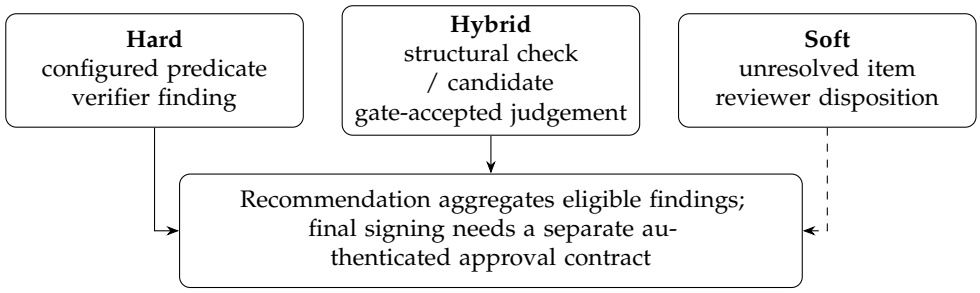


Figure 2. Operational routes and their boundary. A tier is one component of effective authority, not a sufficient permission ceiling for subsequent edits. The complete signing contract is a deployment requirement, not an evaluated capability.

In the default registration path, absence evidence is verifier-owned: a model cannot establish non-detection by inventing a quotation such as "not mentioned". A verifier absence record identifies the expected section and the failed search predicate; it does not establish semantic absence outside the parser's supported input and vocabulary. Hybrid non-detection defaults to pending. Candidate existence is not satisfaction, and lack of a registered finding is not a verified pass.

Package validation rejects inconsistent verifier/tier combinations, missing lexicon references and duplicate identifiers. This static validation applies when packages are loaded. The evaluated update paths can also change no-candidate behaviour, activate a valid hybrid draft, alter severity and reset overrides; Section 3.7 therefore extends validation to the complete effective-authority ceiling after every learned edit. Immutability of the tier field alone is insufficient for this stronger guarantee.

All four blocker rules in this package (CAP-001, SEC-001, SEC-002 and SEC-004) are hybrid; the ten hard rules are major or minor. This distribution motivates tier-specific failure analysis, but is a characteristic of this checklist, not a general inverse relationship between severity and verifiability.

Table 4. Composition of the frozen package used in the retrospective evaluation.

Dimension	Hard	Hybrid	Soft	Total	Blocker	Major	Minor
Capacity	2	3	1	6	1	5	0
Security	0	4	1	5	3	2	0
Interface	2	3	0	5	0	3	2
Documentation	6	0	2	8	0	4	4
Channel	0	1	0	1	0	1	0
Data synchronisation	0	1	0	1	0	1	0
Configuration	0	0	1	1	0	0	1
Monitoring	0	1	0	1	0	0	1
Total	10	13	5	28	4	16	8

3.3. Parsing, Verifiers and Business Predicates

The review engine accepts Markdown. Word and PDF conversion occurs upstream through the host's office tooling; no conversion-fidelity evaluation is reported here. The parser constructs a heading tree, tables and Mermaid sequence diagrams [38], with source offsets. Its normalisation includes NFKC folding, punctuation folding, emphasis removal and whitespace collapsing, with a map back to the source. Chinese and Latin sentence terminators and table-cell boundaries define candidate units. Absence records use an expected section and a zero-length source range.

Table 5. Implemented verifier families. An established parser predicate is not necessarily the complete business predicate. Uses count occurrences in verifier trees.

Family	Uses	Implemented check	Scope limitation
statement	16	Keyword matches, local negation, placeholders, assertion cues	Does not establish semantic sufficiency
section	9	Heading, body length and placeholder checks	Alias and format dependent
table	3	Column groups, row count and placeholder cells	Not correctness of cell content
mermaid-sequence	3	Parseable diagram, participants, messages and required families	Cannot inspect arbitrary embedded images
cross-ref	3	Attachment-reference resolution	Not attachment correctness
quantity	1	Number/unit near a context family	Not validity of the quantity
all, first	4, 2	Conjunction and ordered locating fallback	Inherits child limitations

Negation checks use a window around a matched family; placeholder checks combine lexical tokens and structural cues. These choices recover defects that substring presence misses, but their correctness remains relative to the declared predicate and lexicon. The frozen diagram rules report an image-only diagram as a failure because they require Mermaid. That result supports compliance with a *declared machine-readable format*, not a finding that the depicted design is missing or wrong. Unless an organisation requires that format, unsupported image input should be routed to *manual review* rather than classified as business noncompliance. This correction changes rule behaviour and may require label migration and re-evaluation; the retrospective counts below are not presented as results of such a corrected package.

3.4. Located-Evidence Gate

Model-generated findings are submitted through a registration tool. The gate rejects hard-rule registration, disallows model-produced absence evidence, checks normalised quote length (8–300 characters), locates the quotation and checks its relation to a hybrid candidate. Unknown or soft-rule submissions remain pending rather than gaining automatic eligibility. Rejection reasons are stored for inspection and possible repair.

Algorithm 1: Gate in the evaluated prototype (simplified). Scope membership is overlap, not complete containment; location does not establish entailment.

Input: document, rule id, evidence list, candidate ranges, optional stated section
Output: gate-confirmed finding or pending item with reasons

```
Resolve rule; unknown or soft rule adds a pending reason
if tier is hard then
    Reject model registration
end if
if evidence is empty then
    Add missing-evidence reason
end if
for each evidence item e do
    if e asserts absence or starts with an absence marker then
        Add absence-not-allowed reason
    end if
    Normalise e.quote; add a reason if length is outside 8–300
    Find all source hits of the normalised quote
    Prefer candidate-and-section hit, then candidate hit, then section hit, then first hit
    if no hit exists then
        Add quote-not-found reason
    else
        Attach the selected source range h
        if tier is hybrid and h overlaps no candidate then
            Add out-of-candidates reason
        end if
    end if
end for
return gate-confirmed if no reason; otherwise pending with reasons
```

The gate establishes a recoverable quotation and a positional relationship, not that the quotation supports the allegation. In particular, partial overlap can pass, and a genuine but misleading quotation can be accepted. A strict containment contract would instead require every quoted character to lie within an allowed range and would need separate boundary tests and a new evaluation. The reported 1,031/1,386 fraction therefore measures *gate acceptance*, not semantic evidence validity or prompt-injection resistance.

3.5. Recommendation and Final Disposition

Let C contain findings with an eligible route (hard, hybrid or expert) and status confirmed, adopted or rewritten (the accept action sets the adopted state). Under the frozen policy, gate-confirmed hybrid findings can already enter C before expert adoption. With m the number of minor findings in C and $\theta = 3$,

$$P(C) = \begin{cases} \text{Fail}, & \exists i \in C : \text{severity}(i) = \text{blocker}, \\ \text{Revise}, & \exists i \in C : \text{severity}(i) = \text{major or } m \geq \theta, \\ \text{Pass}, & \text{otherwise.} \end{cases}$$

Pending and soft items do not count; dismissed findings leave C. Opinion-channel findings become numbered clauses; checklist-channel findings form a correction annex, but both eligible channels contribute to the policy. Deterministic aggregation means repeatability for the same finding set; it does not make that set correct, and a model can influence the recommendation through accepted registrations or severity choices.

The prototype supports a *trusted-operator workflow* and records final verdicts through a tool requiring a reviewer-name field. This field provides operator attribution rather than

authenticated approval, and finalisation does not reliably require every pending item to have been disposed of. Suggested/final verdict disagreement may be recorded, but cannot by itself measure rubber-stamping.

3.6. Feedback Records and Configuration Recovery

The review preset prescribes preliminary review, expert disposition and finalisation. It asks the agent to present findings in small batches with quotes and suggested remedies; accept, dismiss, rewrite and add actions become structured records. Statistics and exemplars can subsequently be recalled. The software stores these records for inspection and reuse; longitudinal effects on judgement quality are outside this evaluation.

Table 6. Storage objects and implemented write paths in the prototype; Section 3.7 specifies authenticated approval for deployment.

Object	Contents and write operation	Subsequent use
rule_overrides	Status, severity, suggestions, whitelisted parameters, drafts; update tools	Effective rule loading and rules view
reviews	Input digest, recommendations, final value, timings, displayed version	Resumption and dashboard
issues	Findings, evidence, tier, severity and gate outcome; check/registration/decision	Opinion and evaluation
feedback	Accept/dismiss/rewrite records with reasons	Aggregation
exemplars	Accepted suggestions and rewrites; aggregation	Suggestion context
rule_candidates	Unmatched additions and draft rules	Rule inspection/activation
eval_runs	Configuration results and per-rule outputs	Dashboard and retrospective analysis

A requested severity or status edit can carry a verbatim `basis.quote`, recorded with operator, time and review. An arbitrary sufficiently long string satisfies this check; it does not prove consent. Parameter changes and valid-draft activation can also change decision eligibility while leaving the tier field unchanged. Reset removes overrides and keeps statistics, but may restore stronger shipped settings than the last approved settings.

The displayed identifier `pack@ver+ovN` counts rules with active overrides. Two distinct configurations can have the same N; it is not a content identity and does not enable exact historical replay. Audit records are useful provenance, but reversibility requires retaining the actual configuration to be restored and validating its authority, rather than merely deleting overrides.

3.7. Human-Approval and Recovery Contract for Deployment

For deployment under a software-enforced human-approval claim, we specify the following contract. It is a deployment requirement and is not part of the fixed-corpus evaluation. Effective authority is a tuple of permitted decision subject, admissible evidence, no-candidate behaviour, severity, applicability, activation status and policy eligibility. Every write path must validate the merged configuration against its approved ceiling, including parameter edits, draft activation, finding severity, enable/disable and reset.

A trusted host must issue an approval event after displaying the exact object and action to an authenticated reviewer. The event must bind review/rule identity, input and opinion/configuration content, action and values, principal and consumption or expiry state. An agent-provided name or quotation must not mint that event. Content changes, cross-object use and replay invalidate approval. Final signing requires explicit disposition of pending items or a separately authorised waiver with a reason; the interface must not confuse “Pass on confirmed findings” with review completion.

Recoverable versions require content-addressed immutable snapshots of rules, lexicon, overrides, policy and relevant parser/verifier versions, linked to input and approval records. Restoration must recover the specified approved state and reproduce its expected behaviour on fixed inputs while retaining the audit history. Acceptance testing should cover valid approvals as well as forged, missing, replayed and changed-content approvals; authority-changing parameter/draft/severity/reset paths; unresolved-item signing; partial-overlap quotes; and distinct configurations with the same override count.

This contract assumes trusted host authentication, storage and the controlled review APIs. A malicious administrator, or an agent with arbitrary host shell/file write access, operates outside the boundary that the review layer can defend. Scoped reading may reduce exposed context, but does not establish protection against prompt injection, semantic manipulation or exfiltration [31,37].

3.8. Implementation and Evaluated Version

HARK [39] is delivered as an extension package for a general-purpose conversational agent host, which supplies model access, file conversion, structured questions, delivery, JSON-backed storage and a web server. Hosting is a deliberate design position rather than an implementation shortcut: a review system has to specify which party may retrieve evidence, which may propose a finding, and which may commit a verdict, and anchoring those permissions at a host boundary keeps them explicit and separable from review logic. The engine itself is host-independent: the parser, verifiers, gate, policy, aggregation and renderer are pure functions over rule packages, documents and recorded state, and the engine modules import nothing from the host, so the review semantics can be executed and audited without a particular deployment. The package defines seven storage objects, nine tools, three state-query slash commands and a host-served dashboard while reusing host authentication, file handling and delivery; it requires no separate application backend or embedding index.

Table 7. Principal tool routes and their current boundaries. A prompt instruction is not an authentication mechanism.

Tool	Purpose	Evaluated boundary
`review_parse`	Parse/resume input	Supported text and input digest
`review_check`	Run configured verifiers	Package validation; hard findings/hybrid candidates
`review_record_issue`	Register a judgement	Hard registration rejected; soft/invalid evidence pending; hybrid overlap gate
`review_status`	Show findings and unresolved items	Visibility does not force disposition
`review_decide`	Record disposition/final value	Name is recorded, not authenticated consent
`review_finalize`	Render, close and aggregate	Pending completion is not a reliable enforced prerequisite
`rules_list`, `rules_propose`, `memory_recall`	Inspect, edit/draft and recall	Tier-field restrictions do not cover all effective-authority changes

The retrospective analysis uses released outputs associated with the code repository. A future rule, gate, policy or state-machine repair must be evaluated under a new frozen version; these historical detection scores do not certify it.

Moving to another document domain requires new predicates, source adapters and labelled evaluation, not merely a second YAML file.

4. Results

4.1. Retrospective Evaluation Setup

Corpus and labels.

The project corpus contains 2,000 proposals and 5,323 planted defects over 23 verifiable rules: 1,861 absent, 883 negated, 1,230 placeholder and 1,349 keyword-present-but-empty (hollow). One hundred and thirty-three documents have no planted defect; 1,867 have at least one. Expected verdicts are derived from the planted defects using the frozen policy: 388 Pass, 813 Revise and 799 Fail. Two hundred and fifty-five of the 388 policy-Pass documents contain permissible minor defects, so "policy Pass" must not be used as a synonym for "defect free".

The corpus and phrase pools were developed with the package. Its self-check guards selected execution-consistency conditions, including unexpected verifier failures on unplanted requirements, but does not require every planted defect to produce a verifier failure. Planted defects can remain pending or be missed. The released labels are retained for this retrospective analysis, including the lifecycle and format ambiguities discussed in Section 3.3.

Configurations.

B0 uses rule-specific substrings without the structural and negation handling of the combined system and applies the same aggregation policy to its outputs. S-full uses the verifiers, candidate routes, gate and policy, with a deterministic *flag judge*: it proposes a finding when a candidate carries a negation, assertion-only or placeholder cue. It does not interpret design adequacy. S−verifier widens the hybrid statement search to the whole document. B0 does not invoke this judge; the comparison changes multiple mechanisms and cannot isolate a causal typing effect.

Metrics and analysis provenance.

Alignment is by (document, rule). We recomputed micro-precision, micro-recall and micro-F1 from summed true positives (TP), false positives (FP) and false negatives (FN), and mean per-document F1 from the stored outputs. When both the truth set and the prediction set are empty, per-document F1 is one. We also recomputed policy-verdict accuracy and unweighted Cohen's κ [40,41], gate acceptance, and referral of missed planted defects to pending review. Out-of-package additions and soft-rule items do not enter the 23-rule detection scores.

Table 8. Descriptive results from stored outputs for the frozen configurations on the fixed corpus.

Configuration	TP	FP	FN	Precision	Recall	Micro-F1	Accuracy / κ
B0 keyword	871	2,313	4,452	0.274	0.164	0.205	37.2% / 0.034
S-full	3,165	0	2,158	1.000	0.595	0.746	75.4% / 0.632
S−verifier	3,165	0	2,158	1.000	0.595	0.746	75.4% / 0.632

4.2. Combined-Configuration Trade-offs (RQ2)

Micro-F1 rises from 0.2048 to 0.7458, a difference of 0.5410. Mean per-document F1 rises from 0.1798 to 0.6846, a paired mean difference of 0.5048, which is a different aggregation from the micro-F1 difference. S-full has perfect per-document F1 on 648 documents and median F1 0.800. The stored ablation also reports bootstrap intervals for micro-F1 (S-full 0.746, 95% interval 0.736–0.755; B0 0.205, 0.193–0.215) and paired tests over the 2,000 documents (Wilcoxon signed-rank on per-document F1, McNemar on verdict correctness, Holm-corrected; $p < 10^{-3}$ for both comparisons against S-full). These quantify the separation between the two stored configurations on this corpus.

Table 9. Retrospective detection by operational tier and severity. Grouping identifies where errors occur; it does not establish that the tier caused the result.

	S-full			B0		
Group	TP / FP / FN	Precision	Recall	TP / FP / FN	Precision	Recall
Hard (10 rules)	2,072 / 0 / 414	1.000	0.833	582 / 268 / 1,904	0.685	0.234
Hybrid (13 rules)	1,093 / 0 / 1,744	1.000	0.385	289 / 2,045 / 2,548	0.124	0.102
Blocker	530 / 0 / 600	1.000	0.469	168 / 1,032 / 962	0.140	0.149
Major	1,738 / 0 / 994	1.000	0.636	388 / 997 / 2,344	0.280	0.142
Minor	897 / 0 / 564	1.000	0.614	315 / 284 / 1,146	0.526	0.216

The zero false-positive count is an observed property of these labels and this judge, not a consequence of located evidence. Six hundred of 1,130 blocker defects are missed. All blocker rules happen to be hybrid in this package; neither that composition nor the recall ordering proves a general severity–verifiability relationship. Four hundred and fourteen misses occur in hard rules, showing that automatic eligibility can coexist with inadequate proxy predicates.

4.3. Traceability and Pending Workload (RQ3)

One thousand seven hundred and forty-four of 2,158 missed planted defects occur in pending hybrid document–rule pairs (80.8% referral coverage). Four hundred and fourteen remain unsurfaced by the evaluated automatic routes. Referral is not recovery: the stored result does not show that a reviewer read the item, identified the defect or corrected the verdict. The full pending set contains 18,346 unique hybrid pairs, mean 9.17 per document, median nine and range 4–13, excluding soft-rule items. One thousand seven hundred and forty-four of these 18,346 pairs match planted defects (9.5% label density). The other pairs must not be called false positives: a pending item asserts uncertainty, not a defect. Nor is that density a measure of review time, which was not measured.

Table 10. Routes of planted defects. "Unsurfaced" refers to the evaluated machine routes, not to what a human could find by reading the document. Pending is not a confirmed finding.

Kind (planted count)	Found	Pending	Unsurfaced	Recall
Absent (1,861)	1,169	645	47	0.628
Negated (883)	688	195	0	0.779
Placeholder (1,230)	926	184	120	0.753
Hollow (1,349)	382	720	247	0.283
Total (5,323)	3,165	1,744	414	0.595

The flag judge submits 1,386 hybrid findings. One thousand and thirty-one pass the gate; 355 are rejected, 341 for `QUOTE_TOO_SHORT` and 14 for `ABSENCE_NOT_ALLOWED`. Gate acceptance is 1,031/1,386 (74.4%).

4.4. Policy-Verdict Errors and Sensitivity

Table 11 shows the S-full recommendations against the policy-derived labels.

Table 11. S-full verdict confusion matrix; rows are expected and columns suggested. All 493 disagreements are less severe than expected (under-calls).

Expected / suggested	Pass	Revise	Fail
Pass	388	0	0
Revise	152	661	0
Fail	167	174	458

Here, *policy-non-Pass* denotes documents whose expected verdict is Revise or Fail, and an *under-call* denotes a suggested verdict less severe than the policy-derived label. Under these definitions, 493 of 1,612 policy-non-Pass documents are under-called (30.6%); 319 receive Pass (19.8%), comprising 152 expected Revise and 167 expected Fail. Of 799 expected Fail documents, 167 receive Pass (20.9%) and 341 receive a non-Fail recommendation (42.7%). Three hundred and nineteen of the 707 suggested Pass recommendations are incorrect against the policy labels (45.1%). The 133 genuinely zero-defect documents all have no finding. Two hundred and fifty-five further documents are policy Pass despite minor planted defects. These denominators prevent both inflating "clean-document" performance and mistaking all defects on a Pass document for a false clearance.

With expected labels held fixed under the shipped $\theta = 3$ policy, changing the recommendation's minor threshold gives Table 12. The plateau is conditional on this corpus and finding set, not general robustness or a reason to disregard missed blockers.

Table 12. Recommendation-threshold sensitivity against the fixed original labels, not against labels regenerated under each threshold.

θ	1	2	3 (shipped)	4	5
Accuracy	69%	74%	75%	75%	75%
Cohen's κ	0.525	0.612	0.632	0.632	0.632

4.5. A Historical Walkthrough and Its Label Boundaries

The proposal `syn-hist-30` concerns a balance-query capability opened to third-party channels. Its three gate-confirmed blockers are placeholder concurrency control (CAP-001), a negated binding-relation check (SEC-002), and placeholder exposure-risk assessment (SEC-004). A fourth confirmed finding is the hard diagram rule DOC-001. The stored expected and suggested verdicts are both Fail, and per-document F1 is 0.889. The automated hybrid actor in this corpus case is the flag judge, not an LLM.

Two parts of this case cannot be interpreted as independent evidence of business defects. DOC-001 classifies an image-only diagram as a failure under the frozen Mermaid predicate, without assessing what the image shows. MON-001 treats "New page tracking points must be added to count usage" as lacking an implementation statement. For a proposal before construction, that may instead be an implementation plan whose completeness needs review; lack of completed deployment is not automatically a proposal defect. We retain these historical labels to keep the analysis version-consistent, but identify the task/label discrepancies. Domain-owner adjudication and a new frozen evaluation are required before treating a corrected predicate as validated.

4.6. Development-Team Workflow Evidence (RQ4)

During its first year, the development team applied the workflow to approximately 2,000 operational documents. The observable effect is coverage and traceability rather than a measured accuracy gain: the documents passed through the same typed rule package, each surfaced finding carries a located quote or an explicit absence record, and each reviewer disposition is written back to the record. The workflow therefore leaves an inspectable knowledge state, not merely an individual reviewer's conclusion.

5. Discussion

5.1. Answers to the Research Questions

RQ1: representation, operational authority, and implementation boundaries.

HARK makes review criteria inspectable as rule records with evidence expectations and operational routes. The package contains 10 hard, 13 hybrid, and five soft rules; all

four blocker rules are hybrid. These tiers identify execution paths in this package, not intrinsic decidability of the corresponding business requirements. A successful lexical check establishes its observable predicate, not the adequacy of a load-testing plan or security design. The evaluated snapshot separates verifier-produced findings, gate-confirmed findings, expert adoption, and final disposition in its data model. Gate-confirmed blockers can nevertheless affect suggested verdicts before expert adoption, and some hybrid structural failures are emitted directly by verifiers; Section 3.7 specifies the additional controls required for deployment.

RQ2: fixed-corpus detection and verdict trade-offs.

The combined S-full configuration produces 3,165 true positives, no false positives, and 2,158 false negatives, compared with 871, 2,313, and 4,452 for B0. Micro-F1 increases from 0.205 to 0.746, a difference of 0.541; the mean paired per-proposal F1 difference is 0.505, a distinct aggregation. This is a bounded improvement on a regression corpus. B0 and S-full differ in detectors, cue handling, candidate routing, and evidence checks, and B0 does not use the flag judge. The comparison therefore does not isolate typing's causal effect. Identical S-full and S−verifier results show that this deterministic judge does not expose an effect of wider span scoping, not that scoping is effective or unnecessary for an LLM.

The verdict trade-off is one-sided but not benign. Agreement with policy-derived labels is 75.4%. Of 1,612 policy-non-Pass documents, 493 are under-called and 319 receive Pass: 152 expected Revise and 167 expected Fail. The latter 167 are among the 799 expected Fail documents. The corpus contains 133 zero-defect documents and 1,867 with at least one defect; the 388 policy-Pass documents are a different group, including 255 with minor defects. No finding is raised on the 133 zero-defect documents. These results describe the specified policy and labels, not reliable release decisions in operational use.

RQ3: traceability, gate acceptance, and unresolved workload.

The flag judge submits 1,386 hybrid findings, of which 1,031 pass the gate and 355 fail it, 341 on the quotation-length floor and 14 as absence-style quotations the rule does not accept. Gate acceptance is 74.4%.

Of 2,158 missed defects, 1,744 appear in pending review and 414 remain unsurfaced. The complete pending workload is 18,346 unique hybrid document–rule pairs, a mean of 9.17 per document and median of nine, excluding soft-rule items. Only 1,744 of these pairs match planted defects. This label density is not pending-item precision: pending expresses uncertainty, not an asserted defect. Referral coverage of 80.8% of misses provides no evidence that a reviewer read, resolved, or corrected them. A suggested Pass based on eligible findings is consequently not equivalent to a completed review.

5.2. Implications for Evidence-Bearing Document Review

The useful abstraction is a review record linking a criterion, an evidence object, an operational route, and a disposition. This complements comment-generation approaches and evidence-aware review workflows [8,24]: it makes the inputs to a recommendation inspectable without treating inspectability as correctness. Human approval remains a design requirement whose engineering implementation must be distinguished from ordinary operator-led workflow.

The same abstraction could support assurance cases, compliance dossiers, or architecture designs if each supplies explicit criteria and addressable evidence. Such adaptation would require domain-specific evidence objects, lifecycle assumptions, rule provenance, disposition responsibilities, and separate validation. Narrow model input likewise does not eliminate prompt injection or feedback-storage privacy risks [31,37]. The present evidence

supports the architecture's executable composition and its fixed-corpus trade-offs, while exposing the limits that govern their interpretation.

6. Limitations and Future Work

Judge and component attribution.

The evaluated flag judge provides a deterministic decision procedure based on verifier cues rather than an interpretation of design intent. Its recall and precision are neither general lower nor upper bounds for an LLM, which could both recover semantic defects and introduce unsupported findings. The B0–S-full comparison changes detectors, cue handling, candidate routing, and evidence checks, so it does not isolate typing's individual effect. Cross-model stability, semantic entailment at the gate, and the benefit of span scoping for generative judges require controlled component tests and model-based evaluations.

Approval, authority, recovery, and feedback use.

The evaluated snapshot records operator-attributed decisions, feedback, and override-count version labels within a trusted-operator workflow. It does not provide authenticated approval or comprehensive effective-authority control: pending issues can survive finalisation, and parameter changes, draft activation, finding-level severity edits, and reset can alter authority beyond the tier field. The displayed version label cannot uniquely identify a configuration or reproduce a past review. Section 3.7 specifies approval binding and content-addressed recovery for deployment, but these controls and any longitudinal benefits of feedback are outside the present evaluation.

Domain, lifecycle, and input coverage.

The frozen 28-rule Chinese-language package supports a version-consistent evaluation of one organisation's technical proposal review. The hard tier denotes the snapshot's automatic route and may use a lexical or structural proxy narrower than the business criterion. Requiring Mermaid reflects a parser-visible check; without a governing format requirement, an unreadable image does not prove a defective design. Future-tense implementation plans must likewise be judged against proposal-stage criteria rather than treated as failed implementation acceptance. Conversion fidelity for Word and PDF inputs was not evaluated. Changing predicates, labels, tiers, evidence rules, languages, or document families requires new evidence mapping, revalidation, and, where affected, recomputation.

Human factors.

Pending referral and recorded dispositions make unresolved work inspectable, but referral is not human resolution, and agreement between suggested and final verdicts does not directly measure rubber-stamping. Human disposition quality, time costs, missed pending items, and automation bias remain unmeasured. Evaluation of these outcomes must distinguish provisional recommendation from review completion.

7. Conclusions

HARK represents rule-governed, evidence-bearing document review through explicit criteria, operational authority routes, located evidence, provisional findings, and decision records. Its system contribution is the executable combination of these review objects and interfaces within a host-independent review engine. Technical solution proposals provide the empirical instantiation, while assurance cases, compliance dossiers, and architecture designs remain potential applications.

On the fixed regression corpus, the combined configuration achieves micro-F1 0.746 versus 0.205 for B0, with 3,165 true positives, no false positives, and 2,158 misses. Policy-

verdict agreement is 75.4%; 493 of 1,612 policy-non-Pass documents are under-called, including 319 suggested Pass and 167 Fail-to-Pass cases among 799 expected Fail documents. These results support HARK as a regression-tested review aid rather than an autonomous decision system; deployment-level approval, configuration recovery, and human-factor outcomes require separate validation.

Author Contributions: Conceptualization, Z.Y.; methodology, Z.Y.; software, Z.Y.; validation, Z.Y., K.W. and Q.W.; formal analysis, Z.Y.; investigation, Z.Y.; resources, H.W.; data curation, Z.Y.; writing—original draft preparation, Z.Y.; writing—review and editing, all authors; visualization, Z.Y.; supervision, H.W.; project administration, Z.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable. The study analyses documents and workflow records produced in the routine internal review process of the authors' organisation. No human participants were recruited, no intervention was applied, and no personally identifiable information is reported.

Informed Consent Statement: Not applicable. The three internal proposals and the historical expert opinions are confidential organisational records and are reported only in aggregate; no individual participant data or identifiable personal information are included.

Data Availability Statement: The rule package and evaluation scripts are available in the artefact repository at https://github.com/yyyyyyzt/HARK-Human-Gated-Authority-Typed-Review-Knowledge. An archived release is available at https://doi.org/10.5281/zenodo.23955205 (release v1.0.0). The 2,000-document corpus, its ground-truth labels, manifest and the stored evaluation runs for B0, S-full and S−verifier are deposited in the repository at `paper/data/eval-2000/`. The historical expert opinions and the internal proposals are private and cannot be released; the development-team workflow results are reported only in aggregate.

Generative AI Disclosure: Generative AI tools were used to assist structural revision, language editing, comparative-literature analysis, and cross-checking of the stored evaluation records.

Conflicts of Interest: The authors declare no conflicts of interest. All authors are affiliated with the organisation whose technical proposal review provides the empirical setting of this study; the evaluation uses proposals and aggregated internal workflow records and reports no business outcomes.

References

1. Fagan, M.E. Design and Code Inspections to Reduce Errors in Program Development. *IBM Syst. J.* **1976**, *15*, 182–211. https://doi.org/10.1147/sj.153.0182
2. Laitenberger, O.; DeBaud, J.M. An Encompassing Life Cycle Centric Survey of Software Inspection. *J. Syst. Softw.* **2000**, *50*, 5–31. https://doi.org/10.1016/S0164-1212(99)00073-4
3. European Parliament and Council. Regulation (EU) 2024/1689 of the European Parliament and of the Council Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act). *Off. J. Eur. Union* **2024**, L 1689.
4. ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*; International Organization for Standardization: Geneva, Switzerland, 2018.
5. Génova, G.; Fuentes, J.M.; Llorens, J.; Hurtado, O.; Moreno, V. A Framework to Measure and Improve the Quality of Textual Requirements. *Requir. Eng.* **2013**, *18*, 25–41. https://doi.org/10.1007/s00766-011-0134-z
6. Femmer, H.; Méndez Fernández, D.; Wagner, S.; Eder, S. Rapid Quality Assurance with Requirements Smells. *J. Syst. Softw.* **2017**, *123*, 190–213. https://doi.org/10.1016/j.jss.2016.02.047

7. Chinnam, H. MARV: A Multi-Agent Approach for Validation of Software Requirements Specifications. Master's Thesis, Tampere University, Tampere, Finland, 2025. Available online: https://urn.fi/URN:NBN:fi:tuni-202507297873 (accessed on 30 October 2026).
8. Alsawalqah, H.; Abadleh, A.; Ibrahim, S.; Alqaralleh, B.A.Y.; Aoun, C.; Zreikat, A.I. Evidence-Aware Human-in-the-Loop LLM Review for Requirements-to-Planning Decisions. *Computers* **2026**, *15*, 608. https://doi.org/10.3390/computers15090608
9. Malik, S.; Singh, A.; Azad, A.P. PolicyGuard: From Organizational Policies to Neuro-Symbolic Compliance Review Engines. *arXiv* **2026**, arXiv:2606.32004.
10. Janisar, A.A.; Meidan, A.; Kalid, K.S.b.; Gilal, A.R.; Sarlan, A.B. Security Requirements Engineering: A Review and Analysis. *Computers* **2025**, *14*, 429. https://doi.org/10.3390/computers14100429
11. Zulfigarzada, S.; Gadirli, A.; Karimov, J.; Cerneckas, D.; Rackiene, R.; Azubalis, M. Cybersecurity Requirements and Certification Standards in Industrial Automation Systems: A Systematic Review. *Computers* **2026**, *15*, 364. https://doi.org/10.3390/computers15060364
12. Ferreira-Santos, J.; Pombo, L. The Art Nouveau Path: Requirements Engineering and Traceability for City-Scale In-the-Wild Mobile Augmented Reality Learning Services. *Computers* **2026**, *15*, 243. https://doi.org/10.3390/computers15040243
13. Veizaga, A.; Shin, S.Y.; Briand, L.C. Automated Smell Detection and Recommendation in Natural Language Requirements. *IEEE Trans. Softw. Eng.* **2024**, *50*, 695–720. https://doi.org/10.1109/TSE.2024.3361033
14. Wang, T.; Wang, W.; Yang, Y. AirReq: Automated Requirements Smell Detection and Elimination for Commercial Aircraft Systems. In Proceedings of the 2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW), 2025; pp. 173–180. https://doi.org/10.1109/REW66121.2025.00027
15. Hemmat, A.; Sharbaf, M.; Kolahdouz-Rahimi, S.; Lano, K.; Yassipour Tehrani, S. Research Directions for Using LLM in Software Requirement Engineering: A Systematic Review. *Front. Comput. Sci.* **2025**, *7*, 1519437. https://doi.org/10.3389/fcomp.2025.1519437
16. Dolcetti, G.; Iotti, E. A Dual Perspective Review on Large Language Models and Code Verification. *Front. Comput. Sci.* **2025**, *7*, 1655469. https://doi.org/10.3389/fcomp.2025.1655469
17. Huang, L.; Yu, W.; Ma, W.; Zhong, W.; Feng, Z.; Wang, H.; et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* **2025**, *43*, 42. https://doi.org/10.1145/3703155
18. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; et al. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638
19. Huang, J.; Chen, X.; Mishra, S.; Zheng, H.S.; Yu, A.W.; Song, X.; et al. Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024), Vienna, Austria, 2024.
20. Djouvas, C.; Andreou, C.; Voutsa, M.C.; Tsapatsoulis, N. Model-Contingent Polarity Bias in Large Language Model Annotation: Implications for Semantic Multimedia Personalization. *Computers* **2026**, *15*, 262. https://doi.org/10.3390/computers15050262
21. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 1–45. https://doi.org/10.1145/3641289
22. Garcez, A.d'A.; Lamb, L.C. Neurosymbolic AI: The 3rd Wave. *Artif. Intell. Rev.* **2023**, *56*, 12387–12406. https://doi.org/10.1007/s10462-023-10448-w
23. Galitsky, B.; Rybalov, A. Neuro-Symbolic Verification for Preventing LLM Hallucinations in Process Control. *Processes* **2026**, *14*, 322. https://doi.org/10.3390/pr14020322
24. Pornprasit, C.; Tantithamthavorn, C. Fine-Tuning and Prompt Engineering for Large Language Models-Based Code Review Automation. *Inf. Softw. Technol.* **2024**, *175*, 107523. https://doi.org/10.1016/j.infsof.2024.107523
25. Sun, T.; Xu, J.; Li, Y.; Yan, Z.; Zhang, G.; Xie, L.; et al. BitsAI-CR: Automated Code Review via LLM in Practice. In Proceedings of the Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Trondheim, Norway, 2025. https://doi.org/10.1145/3696630.3728552

26. Tantithamthavorn, K.; Zou, Y.; Wong, A.; Gupta, M.; Wang, Z.; Buller, M.; et al. RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-Based Code Review Automation at Atlassian. In Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2026), 2026; pp. 34–45. https://doi.org/10.1145/3786583.3786851
27. Lin, H.Y.; Liang, M.; Tantithamthavorn, K.; Thongtanunam, P. Is Agentic Code Review Helpful? Mining Developers' Feedback to CodeRabbit Reviews in the Wild. *arXiv* **2026**, arXiv:2607.03316.
28. CodeRabbit. Learnings. CodeRabbit Documentation. Available online: https://docs.coderabbit.ai/knowledge-base/learnings (accessed on 5 November 2026).
29. Gao, T.; Yen, H.; Yu, J.; Chen, D. Enabling Large Language Models to Generate Text with Citations. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), Singapore, 2023; pp. 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398
30. Zhang, J.; Chen, Y.; Commodore-Mensah, Y.; Oberst, M. Verifiable by Construction: Claim-Level Evaluation of Verbatim Citation in Clinical Question Answering. *arXiv* **2026**, arXiv:2609.15964.
31. Shi, J.; Yuan, Z.; Liu, Y.; Huang, Y.; Zhou, P.; Sun, L.; et al. Optimization-Based Prompt Injection Attack to LLM-as-a-Judge. In Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024), Salt Lake City, UT, USA, 2024; pp. 660–674. https://doi.org/10.1145/3658644.3690291
32. Mosqueira-Rey, E.; Hernández-Pereira, E.; Alonso-Ríos, D.; Bobes-Bascarán, J.; Fernández-Leal, Á. Human-in-the-Loop Machine Learning: A State of the Art. *Artif. Intell. Rev.* **2023**, *56*, 3005–3054. https://doi.org/10.1007/s10462-022-10246-w
33. Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; et al. A Survey on Large Language Model Based Autonomous Agents. *Front. Comput. Sci.* **2024**, *18*, 186345. https://doi.org/10.1007/s11704-024-40231-1
34. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; et al. Self-Refine: Iterative Refinement with Self-Feedback. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 2023.
35. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 2023.
36. Zhang, J.; Hu, S.; Lu, C.; Lange, R.; Clune, J. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. *arXiv* **2025**, arXiv:2505.22954.
37. Wang, B.; He, W.; Zeng, S.; Xiang, Z.; Xing, Y.; Tang, J.; et al. Unveiling Privacy Risks in LLM Agent Memory. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025), Vienna, Austria, 2025; pp. 25241–25260. https://doi.org/10.18653/v1/2025.acl-long.1227
38. Mermaid: Generation of Diagrams and Flowcharts from Text. Available online: https://mermaid.js.org (accessed on 30 October 2026).
39. HARK: Human-Accountable Review Knowledge—Artifact Repository for This Paper. Available online: https://github.com/yyyyyyzt/HARK-Human-Gated-Authority-Typed-Review-Knowledge (accessed on 4 November 2026).
40. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educ. Psychol. Meas.* **1960**, *20*, 37–46. https://doi.org/10.1177/001316446002000104
41. Landis, J.R.; Koch, G.G. The Measurement of Observer Agreement for Categorical Data. *Biometrics* **1977**, *33*, 159–174. https://doi.org/10.2307/2529310